COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	8/6/26	**DATE SUBMITTED:**	8/6/26
NAME:	Reyes, Cyrill Joferson B.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 5.1 Queues

1. PROCEDURE OUTPUT

ILO A: Create queue using C++ STL

Definition from the CPP reference documentation for the queue STL template.

queues are a type of container adaptor, specifically designed to operate in a FIFO context (first-in first-out), where elements are inserted into one end of the container and extracted from the other. **queues** are implemented as containers adaptors, which are classes that use an encapsulated object of a specific container class as its underlying container, providing a specific set of member functions to access its elements. Elements are pushed into the "back" of the specific container and popped from its "front".

The underlying container may be one of the standard container class template or some other specifically designed container class. This underlying container shall support at least the following operations:

`empty`

`size`
`front`
`back`
`push_back`
`pop_front`

The standard container classes deque and list fulfill these requirements. By default, if no container class is specified for a particular queue class instantiation, the standard container deque is used.

`(constructor)`	Construct queue (public member function)
`Empty`	Test whether container is empty (public member function)
`Size`	Return size (public member function)
`Front`	Access next element (public member function)
`Back`	Access last element (public member function)
`Push`	Insert element (public member function)
`Emplace`	Construct and insert element (public member function)
`Pop`	Remove next element (public member function)
`Swap`	Swap contents (public member function)

Member Functions

Sample code for Queue STL in C++:

```
// C++ code to illustrate queue in Standard Template Library (STL)
#include <iostream>
#include <queue>

void display(std::queue <int> q)
{
    std::queue <int> c = q;
    while (!c.empty())
    {
        std::cout << " " << c.front();
        c.pop();
    }
    std::cout << "\n";
}

int main()
{
    std::queue <int> a;
    a.push(10);
    a.push(20);
    a.push(30);

    std::cout << "The queue a is :";
    display(a);

    std::cout << "a.empty() :" << a.empty() << "\n";
    std::cout << "a.size() : " << a.size() << "\n";
    std::cout << "a.front() : " << a.front() << "\n";
```



```
    std::cout << "a.back() : " << a.back() << "\n";

    std::cout << "a.pop() : ";
    a.pop();
    display(a);

    a.push(40);
    std::cout << "The queue a is :";
    display(a);

    return 0;
}
```

Task: Create C++ code using Queue STL that will pass an array of students' names to the queue. Make observations on the operations performed and show screenshots of code and output console per operation. Your output must be in table 5-1.

ILO B: Develop C++ code for queues that utilize arrays and linked lists for different internal representations

B.1. Linked List Implementation

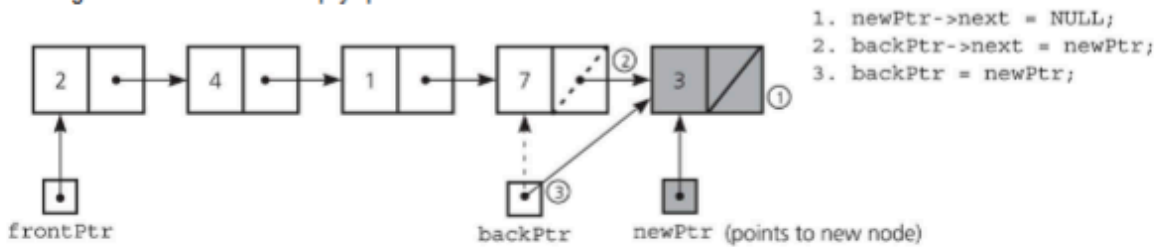
For this section, the linked list implementation will focus on the given behavior of operations as they operate on the nodes and link them together. The nodes are defined the same way as previously, with two compartments: data and pointer to the next.

Operations:

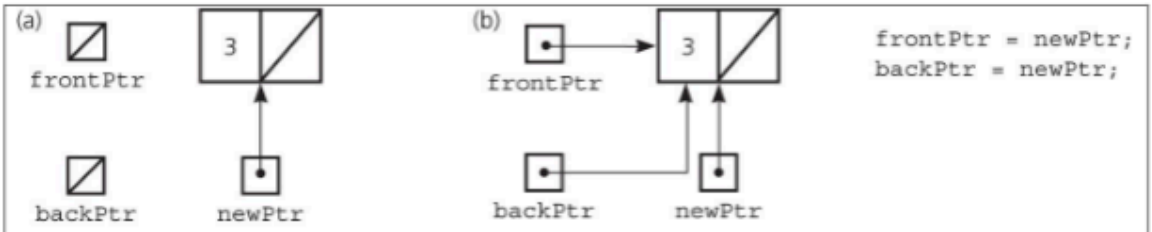
- Inserting an item into a non-empty queue
- Inserting an item into an empty queue
- Deleting an item from a queue of more than one item
- Deleting an item from a queue with one item

For each item, you are provided an algorithm for the operations. **You must implement C++ code for each operation. Then, in section 6 table 5-2, show each operation and its output as shown through the console.**

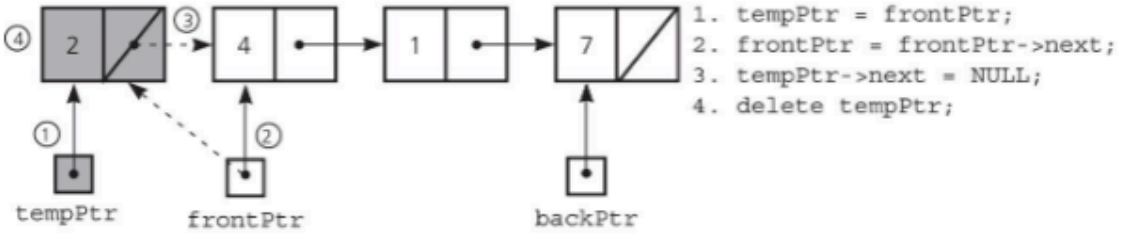
Inserting an item into a nonempty queue



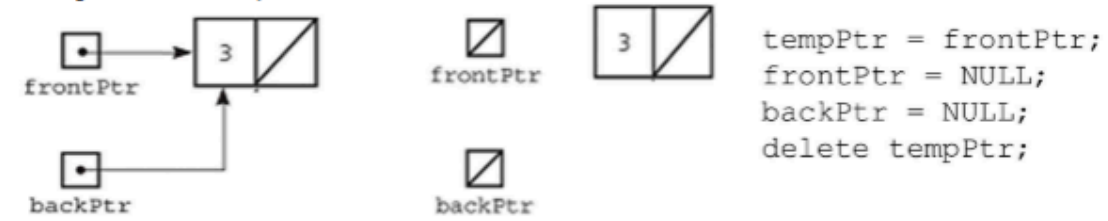
Inserting an item into an empty queue



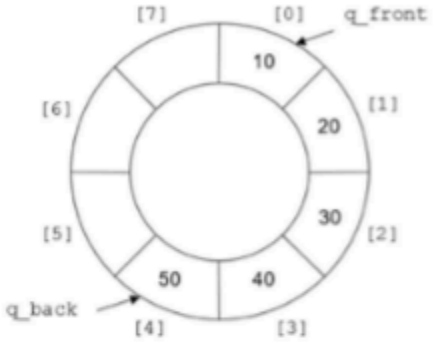
Deleting an item from a queue of more than one item



Deleting an item from a queue with one item



B.2. Array-Based Implementation



In C++, we could implement an array-based queue as a class. To conserve space, we'll implement it as a "circular queue", an array in which the last position is logically connected back to the first position to make a circle. This is sometimes also called a "ring buffer".

Data members

- `q_array` - Queue array pointer. A pointer to the data type of the items stored in the queue; points to the first element of a dynamically-allocated array.
- `q_capacity` - Queue capacity. The number of elements in the queue array.
- `q_size` - Queue size. The number of items currently stored in the queue.
- `q_front` - Queue front. The subscript of the front (or head) item in the queue.
- `q_back` - Queue back. The subscript of the back (or rear or tail) item in the queue.

Member Functions

The example functions described here correspond to the interface of the queue class in the C++ standard library.

`(constructor)`	Construct queue (public member function)
`Empty`	Test whether container is empty (public member function)
`Size`	Return size (public member function)
`Clear`	**Sets the queue size back to 0 and resets q_front and q_back to their initial values. Does not deallocate any dynamic storage or change the queue capacity.**
`Front`	Access next element (public member function)
`Back`	Access last element (public member function)
`Enqueue`	Insert element (public member function)
`Dequeue`	Remove next element (public member function)
`Copy Constructor`	**Similar to the copy constructor for the example Vector class in the notes on dynamic storage allocation. A key difference is that we cannot assume that the items in the queue are stored in elements 0 to q_size - 1 the way we can in the Vector or an array-based stack. It is therefore necessary to copy the entire queue array.**
`Copy Assignment Operator`	**Similar to the copy assignment operator for the example Vector class in the notes on dynamic storage allocation. A key difference is that we cannot assume that the items in the queue are stored in elements 0 to q_size - 1 the way we can in the Vector or an array-based stack. It is therefore necessary to copy the entire queue array.**
`Destructor`	**Deletes the queue array.**

Member Functions (modified from STL) [bold texts are added modifications]

Your Tasks:

- Create a queue class that will contain the array.
- Operations and member functions described in this section must be in your developed code.
- **Test the functions and show the output in table 5-3 in section 6.**

Attribution for 5.2.: http://faculty.cs.niu.edu/~mcmahon/CS241/Notes/Data_Structures/array_based_queue.html

6. Output

Table 5-1. Queues using C++ STL

Table 5-2. Queues using Linked List Implementation

Table 5-3. Queues using Array Implementation

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO A

main.cpp CMakeLists.txt

```cpp
#include <iostream>
#include <queue>
#include <string>

using namespace std;

// Function to display queue contents by value (copies queue so original is unchanged)
static void display(queue<string> q) {
    while (!q.empty()) {
        cout << "\t" << q.front();
        q.pop();
    }
    cout << endl;
}

int main() {
    // Array of student names
    string studentArray[] = {"Cyrill", "Keith", "Clover", "Atomic", "Kitten"};
    int n = std::size(studentArray);

    // Instantiate C++ STL Queue of strings
    queue<string> studentQueue;

    // Enqueue student names from array
    for (int i = 0; i < n; i++) {
        studentQueue.push(studentArray[i]);
    }

```

```
29      // Display initial queue
30      cout << "The queue is :";
31      display( studentQueue);
32
33      // Perform queue operations
34      cout << "a.empty() : " << studentQueue.empty() << endl;
35      cout << "a.size()  : " << studentQueue.size() << endl;
36      cout << "a.front() : " << studentQueue.front() << endl;
37      cout << "a.back()  : " << studentQueue.back() << endl;
38
39      // Pop element (front element removed)
40      cout << "a.pop()   : ";
41      studentQueue.pop();
42      display( studentQueue);
43
44      // Push new element
45      studentQueue.push( x: "Angie");
46      cout << "The queue is :";
47      display( studentQueue);
48
49      return 0;
50  }
```

```
"C:\Users\Joyce Reyes\Documents\CPE21S4 - Reyes\HOA5\main.exe"
The queue is :  Cyrill  Keith   Clover  Atomic  Kitten
a.empty() : 0
a.size()  : 5
a.front() : Cyrill
a.back()  : Kitten
a.pop()   :     Keith   Clover  Atomic  Kitten
The queue is :  Keith   Clover  Atomic  Kitten  Angie

Process finished with exit code 0
```

ILO B

```cpp
#include <iostream>
#include <stdexcept>

using namespace std;

struct Node {
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};

class LinkedListQueue {
private:
    Node* frontPtr;
    Node* backPtr;

public:
    LinkedListQueue() : frontPtr(nullptr), backPtr(nullptr) {}

    ~LinkedListQueue() {
        while (!isEmpty()) {
            dequeue();
        }
    }

    bool isEmpty() const {
        return frontPtr == nullptr;
    }
```

```
29
30      // Algorithm: Inserting an item into a queue
31      void enqueue(int val) {
32          Node* newPtr = new Node(val);
33          if (isEmpty()) {
34              // Algorithm for empty queue:
35              frontPtr = newPtr;
36              backPtr = newPtr;
37          } else {
38              // Algorithm for non-empty queue:
39              newPtr->next = nullptr;
40              backPtr->next = newPtr;
41              backPtr = newPtr;
42          }
43      }
44
45      // Algorithm: Deleting an item from a queue
46      void dequeue() {
47          if (isEmpty()) {
48              cout << "Queue is empty. Cannot dequeue.\n";
49              return;
50          }
51
52          Node* tempPtr = nullptr;
```

```
52          Node* tempPtr = nullptr;
53          if (frontPtr == backPtr) {
54              // Algorithm for queue with 1 item:
55              tempPtr = frontPtr;
56              frontPtr = nullptr;
57              backPtr = nullptr;
58              delete tempPtr;
59          } else {
60              // Algorithm for queue with >1 items:
61              tempPtr = frontPtr;
62              frontPtr = frontPtr->next;
63              tempPtr->next = nullptr;
64              delete tempPtr;
65          }
66      }
67
68      void display() const {
69          if (isEmpty()) {
70              cout << "[ Empty Queue ]\n";
71              return;
72          }
73          Node* curr = frontPtr;
74          while (curr != nullptr) {
75              cout << curr->data << " ";
```

```
            curr = curr->next;
        }
        cout << "\n";
    }
};

int main() {
    LinkedListQueue q;

    cout << "1. Inserting item into empty queue (enqueue 3):\n";
    q.enqueue( val: 3);
    cout << "Queue contents: "; q.display();

    cout << "\n2. Inserting items into non-empty queue (enqueue 7, 1, 4, 2):\n";
    q.enqueue( val: 7);
    q.enqueue( val: 1);
    q.enqueue( val: 4);
    q.enqueue( val: 2);
    cout << "Queue contents: "; q.display();

    cout << "\n3. Deleting item from queue of more than one item:\n";
    q.dequeue();
    cout << "Queue contents: "; q.display();

    cout << "\n4. Deleting items until 1 item remains:\n";
    q.dequeue();
    q.dequeue();
    q.dequeue();
    cout << "Queue contents: "; q.display();

    cout << "\n5. Deleting item from queue with one item:\n";
    q.dequeue();
    cout << "Queue contents: "; q.display();

    return 0;
}
```

```
"C:\Users\Joyce Reyes\Documents\CPE21S4 - Reyes\ILO B\main.exe"
1. Inserting item into empty queue (enqueue 3):
Queue contents: 3

2. Inserting items into non-empty queue (enqueue 7, 1, 4, 2):
Queue contents: 3 7 1 4 2

3. Deleting item from queue of more than one item:
Queue contents: 7 1 4 2

4. Deleting items until 1 item remains:
Queue contents: 2

5. Deleting item from queue with one item:
Queue contents: [ Empty Queue ]

Process finished with exit code 0
```

ILOB.2

main.cpp × CMakeLists.txt

```cpp
#include <iostream>
#include <stdexcept>

using namespace std;

class ArrayQueue {
private:
    int* q_array;
    int q_capacity;
    int q_size;
    int q_front;
    int q_back;

public:
    // (constructor) Construct queue
    ArrayQueue(int capacity = 8) : q_capacity(capacity), q_size(0), q_front(0), q_back(-1) {
        q_array = new int[q_capacity];
    }

    // Copy Constructor
    ArrayQueue(const ArrayQueue& other)
        : q_capacity(other.q_capacity), q_size(other.q_size), q_front(other.q_front), q_back(other.q_back) {
        q_array = new int[q_capacity];
        for (int i = 0; i < q_capacity; ++i) {
            q_array[i] = other.q_array[i];
        }
    }

```

```
29      // Copy Assignment Operator
30      ArrayQueue& operator=(const ArrayQueue& other) {
31          if (this != &other) {
32              delete[] q_array;
33              q_capacity = other.q_capacity;
34              q_size = other.q_size;
35              q_front = other.q_front;
36              q_back = other.q_back;
37              q_array = new int[q_capacity];
38              for (int i = 0; i < q_capacity; ++i) {
39                  q_array[i] = other.q_array[i];
40              }
41          }
42          return *this;
43      }
44
45      // Destructor
46      ~ArrayQueue() {
47          delete[] q_array;
48      }
49
50      // Test whether container is empty
51      bool Empty() const {
52          return q_size == 0;
53      }
```

```
54
55          // Return size
56          int Size() const {
57              return q_size;
58          }
59
60          // Sets queue size back to 0 and resets q_front and q_back
61          void Clear() {
62              q_size = 0;
63              q_front = 0;
64              q_back = -1;
65          }
66
67          // Access next element (front)
68          int Front() const {
69              if (Empty()) throw runtime_error("Queue is empty");
70              return q_array[q_front];
71          }
72
73          // Access last element (back)
74          int Back() const {
75              if (Empty()) throw runtime_error("Queue is empty");
76              return q_array[q_back];
77          }
78
79          // Insert element
```

```
80          void Enqueue(int val) {
81              if (q_size == q_capacity) {
82                  cout << "Queue is full! Cannot enqueue " << val << "\n";
83                  return;
84              }
85              q_back = (q_back + 1) % q_capacity;
86              q_array[q_back] = val;
87              q_size++;
88          }
89
90          // Remove next element
91          void Dequeue() {
92              if (Empty()) {
93                  cout << "Queue is empty! Cannot dequeue.\n";
94                  return;
95              }
96              q_front = (q_front + 1) % q_capacity;
97              q_size--;
98              if (q_size == 0) {
99                  q_front = 0;
100                 q_back = -1;
101             }
102         }
103
```

```
131     cout << "\n2. Dequeue operation:\n";
132     aq.Dequeue();
133     cout << "Queue after Dequeue: "; aq.Display();
134
135     cout << "\n3. Copy Constructor:\n";
136     ArrayQueue aqCopy = aq;
137     cout << "Copied Queue: "; aqCopy.Display();
138
139     cout << "\n4. Copy Assignment Operator:\n";
140     ArrayQueue aqAssign( capacity: 5);
141     aqAssign = aq;
142     cout << "Assigned Queue: "; aqAssign.Display();
143
144     cout << "\n5. Clear operation:\n";
145     aq.Clear();
146     cout << "Is Empty after Clear? " << (aq.Empty() ? "True" : "False") << "\n";
147
148     return 0;
149 }
```

```
"C:\Users\Joyce Reyes\Documents\CPE21S4 - Reyes\ILO B\main.exe"
1. Inserting item into empty queue (enqueue 3):
Queue contents: 3

2. Inserting items into non-empty queue (enqueue 7, 1, 4, 2):
Queue contents: 3 7 1 4 2

3. Deleting item from queue of more than one item:
Queue contents: 7 1 4 2

4. Deleting items until 1 item remains:
Queue contents: 2

5. Deleting item from queue with one item:
Queue contents: [ Empty Queue ]

Process finished with exit code 0
```

ILO C

main.cpp | CMakeLists.txt | Job.h | Node.h | Printer.h | Printer.cpp

```cpp
#include "Printer.h"
#include <iostream>

int main() {
    Printer sharedPrinter;

    std::cout << "=======================================================\n";
    std::cout << "    ILO C: SHARED PRINTER SIMULATION USING QUEUE     \n";
    std::cout << "=======================================================\n\n";

    std::cout << "--- Phase 1: Users submitting initial print requests ---\n";
    sharedPrinter.addJob( id: 101, userName: "Alice", pages: 12);
    sharedPrinter.addJob( id: 102, userName: "Bob", pages: 5);
    sharedPrinter.addJob( id: 103, userName: "Charlie", pages: 25);
    sharedPrinter.addJob( id: 104, userName: "David", pages: 3);

    sharedPrinter.displayQueue();

    std::cout << "--- Phase 2: Printer begins servicing jobs ---\n";
    sharedPrinter.processNextJob();
    sharedPrinter.processNextJob();

    sharedPrinter.displayQueue();

    std::cout << "--- Phase 3: New incoming print request ---\n";
    sharedPrinter.addJob( id: 105, userName: "Eva", pages: 8);

    sharedPrinter.processAllJobs();
```

main.cpp | CMakeLists.txt | Job.h | Node.h | Printer.h | Printer.cpp

```cpp
#include "Printer.h"
#include <iostream>

Printer::~Printer() {
    while (!isEmpty()) {
        processNextJob();
    }
}

bool Printer::isEmpty() const {
    return frontPtr == nullptr;
}

void Printer::addJob(int id, const std::string& userName, int pages) {
    Job newJob(id, userName, pages);
    auto* newNode = new Node(newJob);

    if (isEmpty()) {
        frontPtr = newNode;
        backPtr = newNode;
    } else {
        backPtr->next = newNode;
        backPtr = newNode;
    }
    jobCount++;

    std::cout << "[JOB SUBMITTED] ID: " << id
              << " | User: " << userName
```

main.cpp CMakeLists.txt Job.h Node.h Printer.h Printer.cpp

```cpp
void Printer::addJob(int id, const std::string& userName, int pages) {
                << " | Pages: " << pages
                << " (Pending in queue: " << jobCount << ")\n";
}

void Printer::processNextJob() {
    if (isEmpty()) {
        std::cout << "[PRINTER IDLE] No pending jobs to process.\n";
        return;
    }

    Node* tempPtr = frontPtr;
    Job j = tempPtr->job;

    std::cout << "[PRINTING...] Processing Job ID: " << j.id
              << " | User: " << j.userName
              << " | Printing " << j.pages << " page(s)... [COMPLETED]\n";

    if (frontPtr == backPtr) {
        frontPtr = nullptr;
        backPtr = nullptr;
    } else {
        frontPtr = frontPtr->next;
    }

    delete tempPtr;
    jobCount--;
}
```

```cpp
void Printer::processAllJobs() {
    std::cout << "\n--- Starting Batch Job Processing ---\n";
    if (isEmpty()) {
        std::cout << "Queue is empty. Printer is idle.\n";
        return;
    }

    while (!isEmpty()) {
        processNextJob();
    }
    std::cout << "--- All Print Jobs Successfully Completed ---\n\n";
}

void Printer::displayQueue() const {
    if (isEmpty()) {
        std::cout << "\n[QUEUE STATUS] Queue is empty.\n\n";
        return;
    }

    std::cout << "\n--- Current Pending Print Queue ---\n";
    Node* curr = frontPtr;
    int pos = 1;
    while (curr != nullptr) {
        std::cout << pos++ << ". Job ID: " << curr->job.id
                  << " | User: " << curr->job.userName
                  << " | Pages: " << curr->job.pages << "\n";
        curr = curr->next;
    }
    std::cout << "Total pending jobs: " << jobCount << "\n----------------------------------\n\n";
}
```

main.cpp | CMakeLists.txt | Job.h | Node.h | Printer.h | Printer.cpp

```h
#ifndef JOB_H
#define JOB_H

#include <string>
#include <utility>

class Job {
public:
    int id;
    std::string userName;
    int pages;

    explicit Job(int jobId = 0, std::string name = "", int numPages = 0)
        : id(jobId), userName(std::move(name)), pages(numPages) {}
};

#endif #ifndef JOB_H
```

main.cpp | CMakeLists.txt | Job.h | Node.h | Printer.h

```h
#ifndef NODE_H
#define NODE_H

#include "Job.h"

struct Node {
    Job job;
    Node* next;

    explicit Node(Job j) : job(std::move(j)), next(nullptr) {}
};

#endif #ifndef NODE_H
```

main.cpp | CMakeLists.txt | Job.h | Node.h | Printer.h

```cpp
#ifndef PRINTER_H
#define PRINTER_H

#include "Node.h"
#include <string>

class Printer {
private:
    Node* frontPtr{nullptr};
    Node* backPtr{nullptr};
    int jobCount{0};

public:
    Printer() = default;
    ~Printer();

    Printer(const Printer&) = delete;
    Printer& operator=(const Printer&) = delete;

    [[nodiscard]] bool isEmpty() const;
    void addJob(int id, const std::string& userName, int pages);
    void processNextJob();
    void processAllJobs();
    void displayQueue() const;
};

#endif #ifndef PRINTER_H
```

Table 5-1. Queues using C++ STL

main.cpp | CMakeLists.txt

```cpp
#include <iostream>
#include <queue>
#include <string>

using namespace std;

// Function to display queue contents by value (copies queue so original is unchanged)
static void display(queue<string> q) {
    while (!q.empty()) {
        cout << "\t" << q.front();
        q.pop();
    }
    cout << endl;
}

int main() {
    // Array of student names
    string studentArray[] = { "Cyrill", "Keith", "Clover", "Atomic", "Kitten"};
    int n = std::size(studentArray);

    // Instantiate C++ STL Queue of strings
    queue<string> studentQueue;

    // Enqueue student names from array
    for (int i = 0; i < n; i++) {
        studentQueue.push(studentArray[i]);
    }
```

```
29        // Display initial queue
30        cout << "The queue is :";
31        display( studentQueue);
32
33        // Perform queue operations
34        cout << "a.empty() : " << studentQueue.empty() << endl;
35        cout << "a.size()  : " << studentQueue.size() << endl;
36        cout << "a.front() : " << studentQueue.front() << endl;
37        cout << "a.back()  : " << studentQueue.back() << endl;
38
39        // Pop element (front element removed)
40        cout << "a.pop()   : ";
41        studentQueue.pop();
42        display( studentQueue);
43
44        // Push new element
45        studentQueue.push( x: "Angie");
46        cout << "The queue is :";
47        display( studentQueue);
48
49        return 0;
50    }
```

```
"C:\Users\Joyce Reyes\Documents\CPE21S4 - Reyes\HOA5\main.exe"
The queue is :  Cyrill  Keith   Clover  Atomic  Kitten
a.empty() : 0
a.size()  : 5
a.front() : Cyrill
a.back()  : Kitten
a.pop()   :     Keith   Clover  Atomic  Kitten
The queue is :  Keith   Clover  Atomic  Kitten  Angie

Process finished with exit code 0
```

Table 5-2. Queues using Linked List Implementation

```
1   #include <iostream>
2   #include <stdexcept>
3
4   using namespace std;
5
6   struct Node {
7       int data;
8       Node* next;
9       Node(int val) : data(val), next(nullptr) {}
10  };
11
12  class LinkedListQueue {
13  private:
14      Node* frontPtr;
15      Node* backPtr;
16
17  public:
18      LinkedListQueue() : frontPtr(nullptr), backPtr(nullptr) {}
19
20      ~LinkedListQueue() {
21          while (!isEmpty()) {
22              dequeue();
23          }
24      }
25
26      bool isEmpty() const {
27          return frontPtr == nullptr;
28      }
```

```
29
30      // Algorithm: Inserting an item into a queue
31      void enqueue(int val) {
32          Node* newPtr = new Node(val);
33          if (isEmpty()) {
34              // Algorithm for empty queue:
35              frontPtr = newPtr;
36              backPtr = newPtr;
37          } else {
38              // Algorithm for non-empty queue:
39              newPtr->next = nullptr;
40              backPtr->next = newPtr;
41              backPtr = newPtr;
42          }
43      }
44
45      // Algorithm: Deleting an item from a queue
46      void dequeue() {
47          if (isEmpty()) {
48              cout << "Queue is empty. Cannot dequeue.\n";
49              return;
50          }
51
52          Node* tempPtr = nullptr;
```

```
29
30      // Algorithm: Inserting an item into a queue
31      void enqueue(int val) {
32          Node* newPtr = new Node(val);
33          if (isEmpty()) {
34              // Algorithm for empty queue:
35              frontPtr = newPtr;
36              backPtr = newPtr;
37          } else {
38              // Algorithm for non-empty queue:
39              newPtr->next = nullptr;
40              backPtr->next = newPtr;
41              backPtr = newPtr;
42          }
43      }
44
45      // Algorithm: Deleting an item from a queue
46      void dequeue() {
47          if (isEmpty()) {
48              cout << "Queue is empty. Cannot dequeue.\n";
49              return;
50          }
51
52          Node* tempPtr = nullptr;
```

```
52          Node* tempPtr = nullptr;
53          if (frontPtr == backPtr) {
54              // Algorithm for queue with 1 item:
55              tempPtr = frontPtr;
56              frontPtr = nullptr;
57              backPtr = nullptr;
58              delete tempPtr;
59          } else {
60              // Algorithm for queue with >1 items:
61              tempPtr = frontPtr;
62              frontPtr = frontPtr->next;
63              tempPtr->next = nullptr;
64              delete tempPtr;
65          }
66      }
67
68      void display() const {
69          if (isEmpty()) {
70              cout << "[ Empty Queue ]\n";
71              return;
72          }
73          Node* curr = frontPtr;
74          while (curr != nullptr) {
75              cout << curr->data << " ";
```

```
            curr = curr->next;
        }
        cout << "\n";
    }
};

int main() {
    LinkedListQueue q;

    cout << "1. Inserting item into empty queue (enqueue 3):\n";
    q.enqueue( val: 3);
    cout << "Queue contents: "; q.display();

    cout << "\n2. Inserting items into non-empty queue (enqueue 7, 1, 4, 2):\n";
    q.enqueue( val: 7);
    q.enqueue( val: 1);
    q.enqueue( val: 4);
    q.enqueue( val: 2);
    cout << "Queue contents: "; q.display();

    cout << "\n3. Deleting item from queue of more than one item:\n";
    q.dequeue();
    cout << "Queue contents: "; q.display();

    cout << "\n4. Deleting items until 1 item remains:\n";
    q.dequeue();
    q.dequeue();
    q.dequeue();
    cout << "Queue contents: "; q.display();

    cout << "\n5. Deleting item from queue with one item:\n";
    q.dequeue();
    cout << "Queue contents: "; q.display();

    return 0;
}
```

```
"C:\Users\Joyce Reyes\Documents\CPE21S4 - Reyes\ILO B\main.exe"
1. Inserting item into empty queue (enqueue 3):
Queue contents: 3

2. Inserting items into non-empty queue (enqueue 7, 1, 4, 2):
Queue contents: 3 7 1 4 2

3. Deleting item from queue of more than one item:
Queue contents: 7 1 4 2

4. Deleting items until 1 item remains:
Queue contents: 2

5. Deleting item from queue with one item:
Queue contents: [ Empty Queue ]

Process finished with exit code 0
```

Table 5-3. Queues using Array Implementation

main.cpp CMakeLists.txt

```cpp
#include <iostream>
#include <stdexcept>

using namespace std;

class ArrayQueue {
private:
    int* q_array;
    int q_capacity;
    int q_size;
    int q_front;
    int q_back;

public:
    // (constructor) Construct queue
    ArrayQueue(int capacity = 8) : q_capacity(capacity), q_size(0), q_front(0), q_back(-1) {
        q_array = new int[q_capacity];
    }

    // Copy Constructor
    ArrayQueue(const ArrayQueue& other)
        : q_capacity(other.q_capacity), q_size(other.q_size), q_front(other.q_front), q_back(other.q_back) {
        q_array = new int[q_capacity];
        for (int i = 0; i < q_capacity; ++i) {
            q_array[i] = other.q_array[i];
        }
    }

    // Copy Assignment Operator
    ArrayQueue& operator=(const ArrayQueue& other) {
        if (this != &other) {
            delete[] q_array;
            q_capacity = other.q_capacity;
            q_size = other.q_size;
            q_front = other.q_front;
            q_back = other.q_back;
            q_array = new int[q_capacity];
            for (int i = 0; i < q_capacity; ++i) {
                q_array[i] = other.q_array[i];
            }
        }
        return *this;
    }

    // Destructor
    ~ArrayQueue() {
        delete[] q_array;
    }

    // Test whether container is empty
    bool Empty() const {
        return q_size == 0;
    }

    // Return size
    int Size() const {
        return q_size;
    }

    // Sets queue size back to 0 and resets q_front and q_back
    void Clear() {
        q_size = 0;
        q_front = 0;
        q_back = -1;
    }

    // Access next element (front)
    int Front() const {
        if (Empty()) throw runtime_error("Queue is empty");
        return q_array[q_front];
    }

    // Access last element (back)
    int Back() const {
        if (Empty()) throw runtime_error("Queue is empty");
        return q_array[q_back];
    }

    // Insert element
```

```
    void Enqueue(int val) {
        if (q_size == q_capacity) {
            cout << "Queue is full! Cannot enqueue " << val << "\n";
            return;
        }
        q_back = (q_back + 1) % q_capacity;
        q_array[q_back] = val;
        q_size++;
    }

    // Remove next element
    void Dequeue() {
        if (Empty()) {
            cout << "Queue is empty! Cannot dequeue.\n";
            return;
        }
        q_front = (q_front + 1) % q_capacity;
        q_size--;
        if (q_size == 0) {
            q_front = 0;
            q_back = -1;
        }
    }

    void Display() const {
        if (Empty()) {
            cout << "[ Empty Queue ]\n";
            return;
        }
        for (int i = 0; i < q_size; ++i) {
            int index = (q_front + i) % q_capacity;
            cout << q_array[index] << " ";
        }
        cout << "\n";
    }
};

int main() {
    ArrayQueue aq( capacity: 8);

    cout << "1. Enqueue operations (10, 20, 30, 40, 50):\n";
    aq.Enqueue( val: 10);
    aq.Enqueue( val: 20);
    aq.Enqueue( val: 30);
    aq.Enqueue( val: 40);
    aq.Enqueue( val: 50);
    cout << "Queue: "; aq.Display();
    cout << "Size: " << aq.Size() << "\n";
    cout << "Front: " << aq.Front() << "\n";

    cout << "\n2. Dequeue operation:\n";
    aq.Dequeue();
    cout << "Queue after Dequeue: "; aq.Display();

    cout << "\n3. Copy Constructor:\n";
    ArrayQueue aqCopy = aq;
    cout << "Copied Queue: "; aqCopy.Display();

    cout << "\n4. Copy Assignment Operator:\n";
    ArrayQueue aqAssign( capacity: 5);
    aqAssign = aq;
    cout << "Assigned Queue: "; aqAssign.Display();

    cout << "\n5. Clear operation:\n";
    aq.Clear();
    cout << "Is Empty after Clear? " << (aq.Empty() ? "True" : "False") << "\n";

    return 0;
}
```

```
"C:\Users\Joyce Reyes\Documents\CPE21S4 - Reyes\ILO B\main.exe"
1. Inserting item into empty queue (enqueue 3):
Queue contents: 3

2. Inserting items into non-empty queue (enqueue 7, 1, 4, 2):
Queue contents: 3 7 1 4 2

3. Deleting item from queue of more than one item:
Queue contents: 7 1 4 2

4. Deleting items until 1 item remains:
Queue contents: 2

5. Deleting item from queue with one item:
Queue contents: [ Empty Queue ]

Process finished with exit code 0
```

CONCLUSION:

Working through the different queue implementations really made the First-In, First-Out (FIFO) concept click. While array-based queues are fine when you know the size limit ahead of time, linked lists are much better for handling unpredictable workloads since they scale on the fly. Applying a custom linked list queue to the printer simulation tied everything together, showing how practical queues are for keeping shared tasks organized without missing a beat.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Google Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code styling, Markdown formatting, and output organization.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used Gemini to help organize my C++ code snippets cleanly and format test results into markdown tables. All program design, manual testing, logic verification, and written analysis were completed by me without relying on AI for core content generation.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance

with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.